

Active Directory Home Lab — Phase 2: Attack Simulation & Detection

Domain: Active Directory Security | Penetration Testing | Threat Detection
Tools: Responder · Hashcat · Impacket · bloodhound-python · Kali Linux
Attacks: LLMNR Poisoning · Kerberoasting · Pass-the-Hash · BloodHound Enumeration
MITRE ATT&CK: T1557.001 · T1558.003 · T1550.002 · T1087 · T1069
Phase: 2 of 2 (builds directly on Phase 1 environment)
Date Completed: May 27, 2026

Project Overview

Phase 2 transforms the Active Directory environment built in Phase 1 into an attack simulation lab. A Windows 10 domain-joined workstation (WS01) was added as the primary target, and Kali Linux was reconfigured to the internal lab network as the attacker machine. Four of the most common real-world Active Directory attack techniques were then executed against the apexdefense.local domain each documented with exploitation steps, key findings, Windows Event ID detection signatures, and prevention controls.

The goal of Phase 2 is to demonstrate the attacker's perspective on the same environment administered in Phase 1, showing that the skills to build and manage Active Directory and the skills to attack and defend it are two sides of the same coin.

Lab Network

VM	OS	IP Address	Role
DC01	Windows Server 2022	192.168.1.10	Domain Controller apexdefense.local
WS01	Windows 10 Pro	192.168.1.20	Domain-joined workstation — attack target
Kali Linux	Kali 2025.2	192.168.1.100	Attacker machine

All three VMs communicate over an isolated VirtualBox Internal Network (apexdefense-lab) with no internet access.

Attacks Executed

Attack 1 LLMNR Poisoning

MITRE: T1557.001 | **Tool:** Responder | **Severity:** HIGH

LLMNR (Link-Local Multicast Name Resolution) broadcasts name resolution requests across the local network when DNS fails. Responder intercepts these broadcasts and poisons the response, causing WS01 to authenticate against Kali instead of the legitimate resource. The authentication attempt sends an NTLMv2 hash that can be captured and cracked offline.

What happened:

- Started Responder on Kali: `sudo responder -I eth0 -dwv`
- Triggered LLMNR broadcast from WS01 by navigating to `\\fileshare` in File Explorer
- Responder intercepted and poisoned LLMNR, MDNS, and NBT-NS requests from 192.168.1.20
- NTLMv2 hashes captured for APEXDEFENSE\Administrator and jcarter
- Ran Hashcat mode 5600 (NetNTLMv2) against rockyou.txt
- **Result: Password1 cracked in under 3 seconds. Recovered 1/1 (100%)**

Detection: Event ID 4625 (failed logon), Event ID 4648 (explicit credentials logon). Network monitoring for LLMNR/NBT-NS responses from unexpected hosts.

Prevention: Disable LLMNR via GPO (Turn off multicast name resolution = Enabled). Disable NBT-NS via WINS settings. Enable SMB signing. Enforce strong passwords.

Attack 2 — Kerberoasting

MITRE: T1558.003 | **Tool:** Impacket-GetUserSPNs | **Severity:** HIGH

Any authenticated domain user can request Kerberos service tickets for accounts with registered SPNs. The ticket is encrypted with the service account's password hash and can be cracked offline. Service accounts are prime targets they often have weak passwords, high privileges, and Password never expires enabled.

What happened:

- Ran GetUserSPNs as low-privilege domain user jcarter: `impacket-GetUserSPNs apexdefense.local/jcarter:Password@123 -dc-host DC01.apexdefense.local -request`
- DC01 returned a TGS ticket for sqlservice (SPN: MSSQLSvc/DC01.apexdefense.local:1433)
- Saved hash and ran Hashcat mode 13100 (Kerberos TGS-REP) against rockyou.txt
- **Result: Password1 cracked in under 2 seconds — Recovered 1/1 (100%)**

Detection: Event ID 4769: (Kerberos Service Ticket Request) — specifically RC4 encryption (etype 0x17) indicates Kerberoasting since modern Kerberos uses AES by default.

Prevention: Use Group Managed Service Accounts (gMSA) with auto-rotating 240-character passwords. Audit all SPN-registered accounts. Enable AES encryption for Kerberos tickets. Apply least privilege to service accounts.

Attack 3 — Pass-the-Hash

MITRE: T1550.002 | **Tool:** Impacket-secretsdump + Impacket-psexec | **Severity:** CRITICAL

NTLM authentication uses the password hash directly — the plaintext password is never transmitted. An attacker with a valid NTLM hash can authenticate to any NTLM-accepting system without knowing the password. Secretsdump extracts all domain hashes via the DRSUAPI replication interface using Domain Admin credentials.

What happened:

- Ran secretsdump to dump all domain NTLM hashes: `impacket-secretsdump apexdefense.local/Administrator:[password]@192.168.1.10`
- All 9 domain user hashes, 4 group hashes, sqlservice hash, and computer account hashes (DC01\$, WS01\$) extracted
- Used Administrator NTLM hash to authenticate to WS01: `impacket-psexec Administrator@192.168.1.20 -hashes [LM]:[NT]`
- psexec uploaded service binary to ADMIN\$ share, created and started a Windows service
- Shell returned at C:\Windows\system32>
- Ran `whoami` - **Result: nt authority\system highest Windows privilege level obtained**
- Ran `ipconfig` - confirmed remote code execution inside WS01 at 192.168.1.20

Detection: Event ID 4624 (network logon, NTLM auth type), Event ID 7045 (new service installed). Monitor for ADMIN\$ share access combined with new service creation — classic psexec pattern.

Prevention: Enable Credential Guard to isolate NTLM hashes. Add privileged accounts to Protected Users group. Implement LAPS for unique local admin passwords. Monitor DRSUAPI replication requests from non-DC machines.

Attack 4 — BloodHound Domain Enumeration

MITRE: T1087 + T1069 | **Tool:** bloodhound-python | **Severity:** HIGH

BloodHound collects comprehensive AD data via LDAP queries using any authenticated domain account, then maps relationships between users, groups, computers, GPOs, and ACLs to identify attack paths to Domain Admin. Collection is stealthy — LDAP queries resemble normal AD traffic.

What happened:

- Ran bloodhound-python as jcarter: `bloodhound-python -u jcarter -p Password@123 -d apexdefense.local -dc DC01.apexdefense.local -ns 192.168.1.10 -c all --zip`
- **Collected in under 1 second:** 14 users, 56 groups, 4 GPOs, 7 OUs, 2 computers, 19 containers
- Output: 20260527204317_bloodhound.zip (176KB)
- Analyzed JSON files directly — identified Domain Admins members, computer ACL rights (GenericAll, WriteOwner, WriteDacl), and attack paths
- **Attack path identified:** jcarter → IT-Admins → WS01 admin access → Administrator session → Domain Admins → DC01

Detection: High volume of LDAP queries from a workstation in a short time window. Microsoft Defender for Identity has built-in BloodHound collection detection. Enable DS Access auditing (Event ID 4662) for sensitive AD object access.

Prevention: Deploy Microsoft Defender for Identity. Implement tiered administration model. Clean up excessive ACL permissions. Enable LDAP signing and channel binding.

Full Attack Chain

Step	Attack	Access Gained	Chain Progression
1	LLMNR Poisoning	jcarter plaintext password	Initial credential access via network position
2	Kerberoasting	sqlservice plaintext password	Service account compromise via legitimate Kerberos
3	Pass-the-Hash	SYSTEM shell on WS01	Full workstation compromise without plaintext password
4	BloodHound	Full domain map + attack paths	Visibility to escalate to Domain Admin

Key Troubleshooting — Clock Skew Resolution

The most significant challenge in Phase 2 was a persistent Kerberos clock skew error caused by a 5+ hour UTC offset between Kali and DC01. Kerberos requires clocks within 5 minutes — a security feature preventing replay attacks.

Root causes identified:

- VirtualBox Guest Additions continuously overriding manual Kali time changes
- DC01 Daylight Saving Time disabled — causing 1-hour offset
- DC01 hardware clock interpreted as UTC when Windows expected local time

Resolution: Disabled VBoxManage Guest Additions time sync from Windows host, corrected DC01 DST setting, manually synchronized clocks, added DC01 to Kali /etc/hosts for DNS resolution.

Why this matters: Clock synchronization is a foundational Kerberos requirement. In production, NTP hierarchy failures cause widespread authentication failures across the domain — the same mechanism that protects against replay attacks.

Skills Demonstrated

- LLMNR/NBT-NS poisoning and NTLMv2 hash capture using Responder
 - Offline hash cracking with Hashcat (modes 5600 and 13100)
 - Kerberoasting via Impacket-GetUserSPNs — SPN enumeration and TGS ticket extraction
 - Domain credential dumping via DRSUAPI with Impacket-secretsdump
 - Pass-the-Hash lateral movement and remote code execution via Impacket-psexec
 - Active Directory enumeration with bloodhound-python
 - AD attack path analysis — mapping user to Domain Admin escalation routes
 - MITRE ATT&CK technique mapping for all four attack categories
 - Windows Event ID detection mapping for each attack technique
 - Complex multi-VM troubleshooting — clock synchronization, hypervisor settings, DNS configuration
 - Attack chain documentation for technical and executive audiences
-

Repository Structure

active-directory-home-lab/

```
|
|
|— README.md                ← Phase 1 README
|
|
|— Phase1/
|   |— Bryan_Sykes_AD_Lab_Phase1.docx
|   |— screenshots/
|       |— 01-20 (Phase 1 screenshots)
|
|
|— Phase2/
|   |— README.md            ← This file
```

|— Bryan_Sykes_AD_Lab_Phase2.docx ← Full Phase 2 documentation

|— screenshots/

|— 01-windows10-desktop.png

|— 02-ws01-static-ip.png

|— 03-ws01-domain-joined.png

|— 04-ws01-domain-login-jcarter.png

|— 05-ws01-domain-membership-verified.png

|— 06-ws01-appears-in-ad-computers.png

|— 07-ws01-moved-to-it-ou.png

|— 08-ws01-firewall-disabled.png

|— 09-sqlservice-spn-registered.png

|— 10-kali-network-connectivity.png

|— 11-responder-listening.png

|— 12-llmnr-poisoning-active.png

|— 13-ntlmv2-hashes-captured.png

|— 14-hashcat-exhausted.png

|— 15-new-ntlmv2-hash-captured.png

|— 16-hashcat-password-cracked.png

|— 17-kerberoast-hash-captured.png

|— 18-kerberoast-hashcat-exhausted.png

|— 19-kerberoast-cracked.png

|— 20-secretsdump-hashes.png

- |— 21-secretsdump-complete.png
 - |— 22-pass-the-hash-shell.png
 - |— 23-pass-the-hash-whoami.png
 - |— 24-pass-the-hash-ipconfig.png
 - |— 25-bloodhound-collection-complete.png
 - |— 26-bloodhound-domain-analysis.png
-

Notes

- All attacks executed against a personal isolated lab environment — no live systems targeted
 - apexdefense.local is a fictional domain built specifically for this project
 - Weak passwords used in some accounts for successful crack demonstration — clearly documented in report
 - BloodHound GUI analysis not available due to Neo4j not being installed on isolated network — JSON analysis performed directly as equivalent demonstration
 - Phase 1 snapshot taken before Phase 2 to enable clean rollback
-

Part of an ongoing cybersecurity portfolio. Additional projects cover AWS GuardDuty cloud threat detection, OWASP Top 10 web application exploitation, AI model supply chain risk assessment, Salesforce vendor risk assessment, and Security Onion SIEM detection engineering.